

Demo Guide

Step-by-step script for presenting the complete NexLink project

Item	Detail
Presenter	Maitrey Bharambe
Prepared on	19 October 2026
Demo length	About 25 minutes + questions (9 acts; acts marked optional can be skipped)
Equipment	One Windows PC with this repository, 3–4 terminal windows, a browser
Readiness	All components checked on 19 Oct 2026 — see section 1

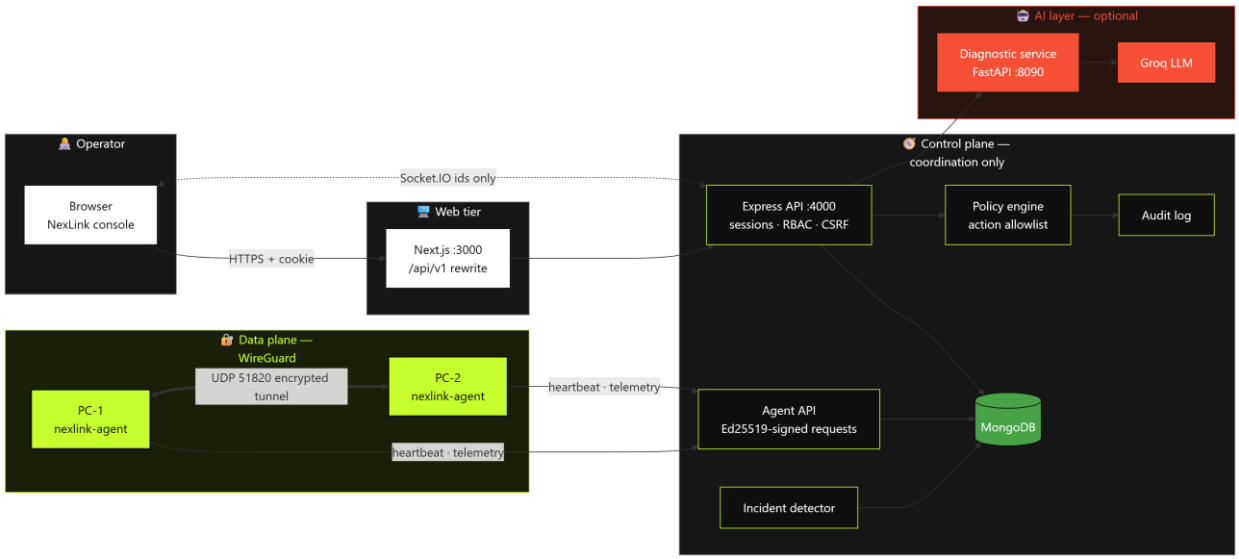
Contents

Demo at a glance 3
1. Readiness check (19 October 2026) 4
2. The day before 5
2.1 Update and install 5
2.2 Use a clean demo database 5
2.3 Decide on AI (Act 7) 5
2.4 Rehearse once 5
3. Starting everything (10 minutes before) 6
Quick checks 6
4. The demo, act by act 7
Act 1 — Introduction and architecture (3 min) 7
Act 2 — Sign up and register two PCs (4 min) 7
Act 3 — Networks and validation (2 min) 7
Act 4 — Honest health (2 min) 8
Act 4b — Protocol simulation (3 min) 8
Act 5 — A real WireGuard tunnel (4 min) 8
Act 5b — A second physical PC on the same Wi-Fi (optional, 4 min) 9
Act 6 — Diagnostics, approval, verification (5 min) 10
Act 7 — AI with guardrails (optional, 2 min) 11
Act 8 — Security: revoke and audit (2 min) 11
Act 9 — Quality (optional, 1 min) 11
5. Questions you may be asked 12
6. If something goes wrong, and resetting 13
Troubleshooting during the demo 13
Resetting for another run 13
Documents to have ready 13
7. Printable checklist 15

Demo at a glance

The story you are telling: NexLink is a private-network platform that only reports what it can prove. Every act shows one idea, in this order:

Act	What you show	Time	Key message
1	Introduction and architecture	3 min	Control plane coordinates; VPN traffic goes device-to-device over WireGuard
2	Sign up and register two PCs	4 min	Devices have their own cryptographic identity; tokens are one-time
3	Networks and validation	2 min	Address space is validated; overlaps and bad ranges are rejected
4	Honest health	2 min	“Enrolled” is not “connected” — no evidence means “unknown”
4b	Protocol simulation	3 min	The exact packet sequence, slowed down; data plane survives a server outage
5	A real WireGuard tunnel	4 min	Connected only after a real handshake; measured through the tunnel
5b	A second physical PC (optional)	4 min	Browser + one download; tunnel PC-to-PC, no admin on PC-2
6	Diagnostics → approval → verification	5 min	Evidence, policy engine, human approval, verified result
7	AI with guardrails (optional)	2 min	AI proposes; rules and humans decide
8	Security: revoke + audit log	2 min	A revoked device is locked out instantly; everything is audited
9	Quality: live tests (optional)	1 min	102 automated tests across three languages



The architecture you explain in Act 1 (also in README and docs/architecture.md).

1. Readiness check (19 October 2026)

Everything needed for this demo was checked on this PC. All checks passed.

Component	Check	Result
Control plane	npm test	**64 / 64 passed**
Networking agent (Go)	go vet + go test ./...	**23 / 23 passed**
Diagnostic service	python -m pytest -q	**15 / 15 passed**
Web console	Production build (npx next build)	**Passed**
Agent binary	networking-agent\bin\nexlink-agent.exe version	**0.2.0, rebuilt 19 Oct**
Real WireGuard tunnel	nexlink-agent lab (earlier full runs)	**Handshake + 0.23 ms verified**
Test-incident peer picker	Console dialog lists only same-network peers	**Works (added today)**
Agent console + device buttons	Live agent output on device page; test tunnel → approve → verified	**Works (0.32 ms, 0% loss)**
Background service mode	Service code run with --debug; log file + console output	**Works (install needs Admin)**
Portal from another PC	http://192.168.0.105:3000 — sign-in, API, agent download, live socket	**Works**
Tunnel between two agent processes over the LAN	run --userspace on UDP 51820 / 51821, LAN address 192.168.0.105	**Handshake + 0.3–2.9 ms, connected · direct**
Database	Old test devices and incidents removed	**Clean: network nexlink-lan with pc-1**
Toolchain	Node 22, Go 1.27 (in .tools), Python 3.13 venv	**Present**
MongoDB	npm run dev:mongo	**Works**
WireGuard for Windows app	Not installed	**Not needed for this demo**
Docker Desktop	Fails to start (stale socket file)	**Not needed for this demo**

Be careful

Use a fresh database for the demo. The development database is now clean (network nexlink-lan with pc-1), but Act 2 shows sign-up from an empty system — Section 2 shows the one-line change. Offline devices and their open incidents look like a broken system on screen.

Acts 5–6 run on one PC (two agents, real WireGuard protocol). If a second PC is on the same Wi-Fi, the optional Act 5b connects it for real — same agent, run --userspace, no admin on that PC.

2. The day before

2.1 Update and install

Open PowerShell in the project folder (D:\projects\cn):

```
git pull
cd control-plane;      npm install;  cd ..
cd frontend;           npm install;  cd ..
cd diagnostic-service; .venv\Scripts\pip install -r requirements.txt;  cd ..
```

2.2 Use a clean demo database

Open `control-plane\.env` and change only the database name at the end of `MONGODB_URI`:

```
MONGODB_URI=mongodb://127.0.0.1:27017/nexlink_demo
```

A new, empty database is created automatically. The first account you sign up becomes the administrator (needed for Act 6). To go back to your old data later, change the name back to nexlink.

2.3 Decide on AI (Act 7)

- **With Groq:** get a key at console.groq.com/keys. Copy `diagnostic-service\.env.example` to `diagnostic-service\.env` and set the two lines below. The file is git-ignored, so the key never reaches GitHub. Also keep `DIAGNOSTIC_SERVICE_URL=http://127.0.0.1:8090` in control-plane\.env.

```
NEXLINK_DIAG_PROVIDER=groq
GROQ_API_KEY=gsk_...your key...
```

- **Without Groq:** either run the service with the mock provider (answers are labelled “mock”), or put a # in front of DIAGNOSTIC_SERVICE_URL to hide the “ask ai” button.

2.4 Rehearse once

- Run the whole guide once from start to finish with the clean database, then reset it (section 6) so the real demo starts empty.
- Set the browser zoom to 80–90% so whole pages fit on the projector.
- Close Word (an open report file creates lock files), Docker Desktop and anything using ports 3000, 4000, 8090 or 27017.
- Keep these open in browser tabs: the GitHub README, and docs/NexLink-Project-Report.docx (or its PDF) for questions.

3. Starting everything (10 minutes before)

Open four PowerShell windows. Give each a clear title so you do not mix them up in front of the audience.

Window 1 — Database

```
cd D:\projects\cn\control-plane
npm run dev:mongo
# wait for: dev mongod listening: mongodb://127.0.0.1:27017/
```

Window 2 — Control plane

```
cd D:\projects\cn\control-plane
npm run dev
# wait for: "control plane listening" (port 4000)
```

Window 3 — Web console

```
cd D:\projects\cn\frontend
npm run dev
# wait for: Ready — then open http://localhost:3000
```

Window 4 — Agents (run as Administrator)

```
cd D:\projects\cn\networking-agent\bin
.\nexlink-agent.exe version     # prints 0.2.0
```

Tip — no agent window during normal use: after enrolling, `.\nexlink-agent.exe install --state-dir <folder>` (Administrator) runs the agent as a Windows service that starts with the PC; its output still appears in the agent console. For the demo itself, the lab command below is clearer.

Window 5 (optional) — AI diagnostic service

```
cd D:\projects\cn\diagnostic-service
# reads the Groq key from diagnostic-service\.env automatically
.venv\Scripts\python -m uvicorn app.api.main:app --host 127.0.0.1 --port 8090
# check http://127.0.0.1:8090/healthz shows "provider":"groq:openai/gpt-oss-120b"
```

Quick checks

- http://localhost:4000/healthz shows `{"ok":true,"service":"control-plane"}`
- http://localhost:3000 shows the NexLink sign-in page with the logo.
- If a page shows “HTTP 500” right after starting, the control plane was still starting — refresh.

4. The demo, act by act

Act 1 — Introduction and architecture (3 min)

1. Show the GitHub README (logo, features table, architecture diagram).
2. Point to the architecture diagram: browser → console → control plane → MongoDB; agents on each PC; the thick lime link between PCs.
3. Mention the stack: Next.js console, Node.js + MongoDB control plane, Go agent, Python AI service.

What to say

"NexLink lets you create private networks between your own computers, see their real connectivity, diagnose problems and run approved actions."

"The control plane only coordinates — keys, addresses, policy. VPN traffic goes directly between the devices over WireGuard and never passes through the server."

"The key design rule: the system never claims something it cannot prove. A device is 'connected' only after WireGuard reports a real handshake."

Act 2 — Sign up and register two PCs (4 min)

1. Open `http://localhost:3000` → click the **sign up** tab.
2. Enter your name, email and a password (12+ characters) → create account. You land on "register this pc".
3. Device name: `demo-a` → **register this pc**. Point out the new private network (e.g. 10.100.0.0/24), the one-time token and its countdown.
4. **copy token**, then in Window 4:

```
.\nexlink-agent.exe enroll --server http://localhost:4000 --state-dir .\demo-a
# paste the token at the prompt, press Enter
```

1. Switch back to the browser: the page changes to "demo-a is in." with its virtual IP.
2. Click **register a pc** in the sidebar → name `demo-b` → register → copy token, then:

```
.\nexlink-agent.exe enroll --server http://localhost:4000 --state-dir .\demo-b
```

What the audience sees

Each enrollment prints the device id, network, address and identity fingerprint, and the browser flips to "registered" by itself (live updates).

What to say

"The agent generated its own keys on the PC — an Ed25519 identity key and a WireGuard key. The private keys never leave the device."

"The token is single-use, expires in 15 minutes and is stored only as a hash. The agent also signs the enrollment to prove it owns the key."

"From now on every request the agent makes is signed, time-stamped and replay-protected."

Act 3 — Networks and validation (2 min)

1. Sidebar → **networks** → open your network: show the address range, assignable hosts and the two devices with their addresses.
2. Back to networks → **new network**. Try these one by one and show each error:

Enter CIDR	Error shown	Point to make
10.100.0.0/25	CIDR_OVERLAP	Overlaps your existing network

Enter CIDR	Error shown	Point to make
10.200.0.5/24	CIDR_NOT_ALIGNED (suggests 10.200.0.0/24)	Host bits must be zero
8.8.8.0/24	CIDR_NOT_PRIVATE	Only private address space allowed
10.150.0.0/24 (name: lab-net)	Created	Valid range accepted

> **What to say**
> "Address allocation is enforced by a unique index in the database, so two devices can never get the same address — even if they enroll at the same moment."

Act 4 — Honest health (2 min)

1. **devices**: demo-a and demo-b show status **unknown**, heartbeat **never**.
2. **topology**: the two devices are joined by a dotted gray "configured" line, not a lime one.
3. Open demo-a: three separate cards — agent ↔ control plane, WireGuard interface, peer tunnels.

> **What to say**
> "They are enrolled, but nothing has proven they can talk, so NexLink says 'unknown' — not healthy, not zero."
> "Health is three independent signals. A heartbeat proves only that the agent reaches the server — it says nothing about the tunnel."

Act 4b — Protocol simulation (3 min)

Explain how the tunnel works before showing the real one. The simulation replays exactly what the agents do, slowed down so it can be followed.

1. Sidebar → **simulation**. Scenario **normal tunnel**, speed 1×. Point to the **SIMULATED PACKETS** badge.
2. Narrate the phase chips as they light up: **enrollment → peer config → handshake → encrypted traffic → telemetry**.
3. During the handshake, pause (space bar) and show the **packet inspector**: Handshake Initiation, 148 bytes, Noise_IKpsk2, encrypted static key and timestamp.
4. Note that the tunnel turns lime as soon as the handshake completes, but the server's **tunnel link** badge only says connected after telemetry arrives.
5. Switch to **control plane down** and click phase **6 scenario event**: the server goes red and telemetry fails, but probes keep flowing through the tunnel.
6. Optional: **peer offline** (handshake retried every 5 s, never marked connected) or **device revoked** (403 DEVICE_REVOKED, peer removed).

> **What to say**
> "This is a replay of our protocol, not live traffic — every packet here is labelled simulated. The message types, sizes and endpoints are exactly the ones our agents use."
> "Notice the two separate states: the PCs have a session after the handshake, but the server only calls it connected once an agent reports evidence."
> "If the server goes down, the tunnel keeps working: the keys are already on both PCs. That is why control plane and data plane are separate."
> "Now let me show you the same thing for real." → Act 5.

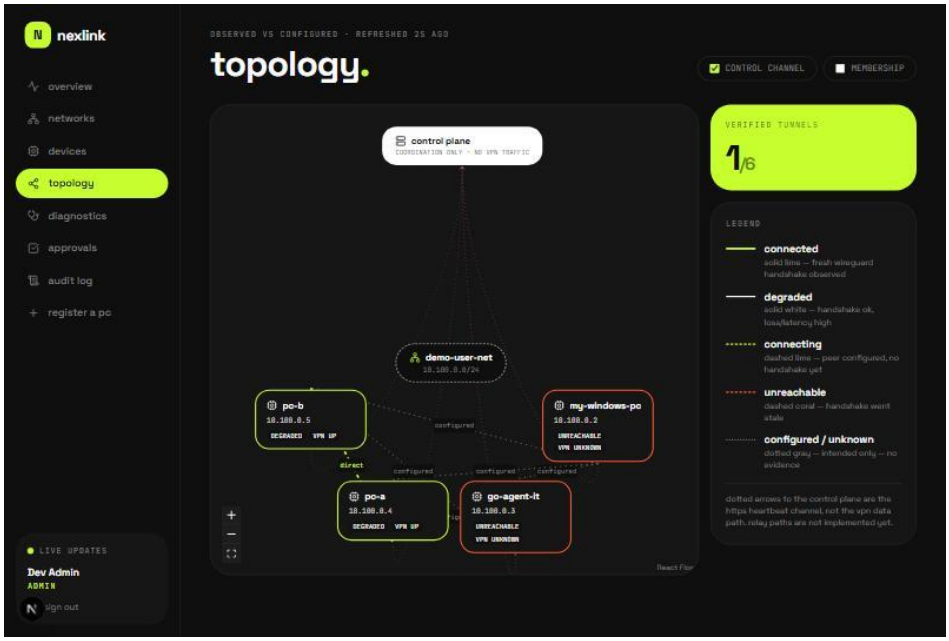
Act 5 — A real WireGuard tunnel (4 min)

In Window 4 start both agents with real WireGuard devices (leave it running until Act 8):

```
.\nexlink-agent.exe lab --server http://localhost:4000 --a .\demo-a --b .\demo-b --duration 0
```

> **Be careful**
> Windows may show a firewall prompt for nexlink-agent.exe — choosing Cancel or Allow both work (traffic stays on this PC).

1. Window 4 prints every 10 seconds: handshake age and tunnel RTT (well under 1 ms) for A and B.
2. **topology** (wait ~20 s): the demo-a ⇄ demo-b line turns **solid lime, labelled "direct"**; "verified tunnels 1/1".
3. **devices → demo-a**: agent online, WireGuard interface up (source wireguard-go), peer tunnels 1/1 with a fresh handshake; status **healthy**.
4. Scroll down on demo-a: the **agent console** streams the agent's own output live — "lab tunnel starting", "telemetry sent", "tunnel status … tunnel_rtt=0.4 ms". Say: "this is the agent on the PC talking to us — no terminal needed."



What the topology looks like once the handshake is observed.

> **What to say**
> "This is the real WireGuard protocol — wireguard-go is the official implementation. Each agent has its own encrypted device; they did a Noise handshake and the RTT you see is measured through the tunnel."
> "The line turned lime only because WireGuard reported a handshake — not because a record exists."
> "For this act both devices run on this PC. Next I connect a real second computer with exactly the same agent."

Act 5b — A second physical PC on the same Wi-Fi (optional, 4 min)

The day before, on this PC (Administrator, once): `.\scripts\lan-firewall.ps1 -Server -Agent` in D:\projects\cn. Wi-Fi must be a Private network. The script prints this PC's LAN address (below: 192.168.0.105). `control-plane\.env` needs `CORS_ALLOW_PRIVATE_LAN=true`.

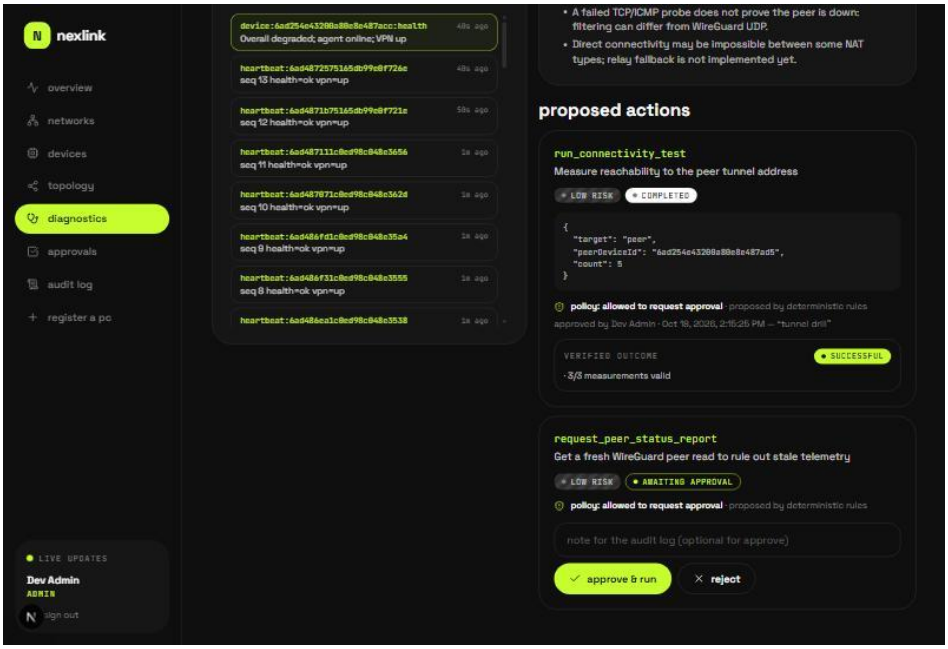
1. On PC-2 open `http://192.168.0.105:3000` and sign in with the **same account** (so PC-2 joins the same network).
2. **register this pc** → name `demo-c` → register → **copy command**.
3. Open PowerShell on PC-2 (no admin), paste, press Enter. The window shows **[1/3] downloading → [2/3] joining → [3/3] starting the tunnel**.

4. Windows asks about network access → Allow. The setup page on PC-2 flips to “demo-c is in.”
5. On this PC, stop the lab (Ctrl+C) and run demo-a as a normal agent so it can reach PC-2: `.\nexlink-agent.exe run --userspace --allow-http --server http://192.168.0.105:4000 --state-dir .\demo-a`
6. **topology** (~30 s): demo-a ⇄ demo-c turns solid lime “direct”. The agent console of demo-c shows “tunnel peers updated” and “tunnel probe … rtt_ms=…”.
7. devices → demo-a → **test tunnel** to demo-c → approve & run → verified result measured between two physical PCs.

> **What to say**
>
> “PC-2 only needed a browser and one pasted line. The agent makes its own keys there; the WireGuard tunnel runs inside the agent, so no VPN app and no admin rights.”
>
> “The portal and the server stay on this PC — the probe traffic you see goes directly PC-to-PC over encrypted UDP.”

Act 6 — Diagnostics, approval, verification (5 min)

1. **diagnostics** → **test event** → device `demo-a`, category **peer handshake stale**, peer device `demo-b`, note “demo drill” → create.
2. Open the new [TEST] incident → run diagnostics.
3. Walk through: evidence gathered by read-only tools → observations (each cites an evidence id) → hypotheses (marked untested/plausible) → uncertainties.
4. Proposed actions: on **run_connectivity_test** click **approve & run**.
5. Wait 10–20 s: the action shows **VERIFIED OUTCOME: SUCCESSFUL** — 3/3 measurements valid (RTT, jitter, 0% loss through the tunnel).



The approved connectivity test, executed by the agent through the tunnel and verified.

Optional 6a — the same thing straight from the device page

1. devices → demo-a → **run on this device**: choose peer `demo-b` → **test tunnel**.
2. The request appears below the buttons as awaiting approval → **approve & run**.
3. Watch the agent console: “executing approved command” then “command finished … rtt_ms=… packet_loss_percent=0”; the card shows **VERIFIED OUTCOME: SUCCESSFUL**.

Optional 6b — the policy engine says no

1. test event → device demo-a, category **vpn interface down** → run diagnostics.

2. The action **request_vpn_config_refresh** shows **rejected by policy** — “agent does not advertise wireguard_apply (privileged mode disabled)”.

> **What to say**
>
> “Diagnosis only uses read-only tools. Every proposed action goes through a deterministic policy engine — allowlist, parameters, device state, rate limits — and then needs a human to approve.”
>
> “The agent checks again with its own allowlist, runs the test, and the server verifies the result from fresh measurements. Test incidents are never auto-resolved.”
>
> “Changing a PC’s network settings needs privileged mode switched on locally on that PC — the server cannot force it.”

Act 7 — AI with guardrails (optional, 2 min)

Only if Window 5 is running.

1. Diagnostics page: point to the cards “read-only tools” and “action allowlist”.
2. Open the incident from Act 6 → **ask ai**. The diagnosis appears labelled with the provider (e.g. groq:openai/gpt-oss-120b, or “mock”).
3. Show that its observations cite evidence ids, and that its proposed actions still show policy verdicts and need approval.

> **What to say**
>
> “The AI only receives evidence summaries — never IP addresses. If it cites evidence that doesn’t exist, adds a confidence score or proposes an action outside the allowlist, the answer is rejected.”
>
> “The AI can suggest; the rules and a human decide. VPN connectivity never depends on the AI being available.”

Act 8 — Security: revoke and audit (2 min)

1. **devices → demo-b** → **revoke** → reason “lost laptop” → revoke.
2. Window 4 immediately prints **agent B stopped … DEVICE_REVOKED**; in topology the demo-b link disappears.
3. **audit log**: search “device” — show device.enroll, device.revoke, action.approve, action.verify, and a denied agent.auth entry.
4. Press Ctrl+C in Window 4 to stop the lab.

> **What to say**
>
> “Revocation takes effect on the device’s very next request — its signature is no longer accepted.”
>
> “Every security-sensitive event is written by the backend to an append-only audit log; there is no API to edit or delete it.”

Act 9 — Quality (optional, 1 min)

In a spare window (takes about a minute):

```
cd D:\projects\cn\control-plane
npm test                    # 64 tests
cd ..\networking-agent
..\.tools\go\bin\go.exe test ./...    # 23 tests
```

> **What to say**
>
> “There are 102 automated tests across Node.js, Go and Python — including a test that brings up a real WireGuard tunnel and checks encrypted traffic, plus an integration test of the real agent against the running server.”

5. Questions you may be asked

Question	Answer
Why doesn't VPN traffic go through your server?	It would be a bottleneck and a privacy risk. The control plane only distributes keys, addresses and policy; WireGuard connects devices directly (data plane).
Is this real WireGuard or a simulation?	Real protocol. The agent embeds wireguard-go, the official userspace implementation: run --userspace builds the tunnel PC-to-PC with no VPN app or admin rights. With the WireGuard app installed, the same agent instead reads the kernel/WireGuardNT interface.
Is the simulation page real traffic?	No — it is a replay of the protocol, and every packet is labelled simulated. It uses the real message types, sizes and endpoints; with two enrolled devices selected it shows their real names and addresses, plus the link state their agents actually report. The real tunnel is on topology and the device pages.
Does the userspace tunnel carry all my traffic?	No — it carries NexLink traffic (handshakes, probes, approved tests). A system-wide VPN adapter uses the WireGuard app path (docs/two-pc-test.md).
How do you stop a device pretending to be another?	Each device has an Ed25519 key generated on the device. Every request is signed over method, path, time, a one-time nonce, device id and body hash. The device id comes from the signature, not the request body.
How do you know a device is healthy?	Three separate signals: agent heartbeat, WireGuard interface state, and a fresh handshake with each peer. Missing evidence shows as unknown, never healthy.
What does the AI actually do?	It reads evidence collected by read-only tools and proposes a diagnosis and actions. Its output must match a strict schema, cite real evidence and use allowlisted actions; a policy engine and a human approve anything that runs.
What if the AI is wrong or down?	Wrong output is rejected by validation; deterministic rules work without AI; networking never depends on it.
Why MongoDB?	Flexible documents for telemetry and incidents, while unique, partial and TTL indexes enforce integrity (unique addresses, one-time nonces) and automatic data retention.
What about PCs behind different home routers?	Direct WireGuard works when one side can receive UDP. For strict NATs a relay is needed; the design is documented in docs/network-protocols.md and is the next networking step.
How are passwords and sessions protected?	scrypt hashing, HttpOnly SameSite=Strict cookies, CSRF tokens, lockout and rate limits; roles enforced in the backend.
How would you deploy it?	One server with Docker Compose and Caddy (HTTPS) for console and API, MongoDB Atlas for data, and agent downloads on GitHub Releases.
How did you test it?	102 automated tests (64 Node.js, 23 Go, 15 Python), an agent-to-server integration test, and real WireGuard tunnel runs — on one PC and between two agents over the LAN.

6. If something goes wrong, and resetting

Troubleshooting during the demo

What you see	What to do
Sign-in shows "HTTP 500" or "control plane unreachable"	Window 2 is still starting or stopped — wait for "control plane listening", then refresh.
ENROLLMENT_TOKEN_EXPIRED	Tokens last 15 minutes — click "register a pc" again for a new one.
"already enrolled as device …"	That --state-dir is already used — pick a new folder name (e.g. .\demo-c).
lab: "control plane does not list agent B as a peer"	Both devices must be registered by the same account (same network) and not revoked.
Topology line stays dotted	Wait 20 seconds and refresh; check Window 4 shows a handshake age.
"live updates offline" in the sidebar	Refresh the page; data still refreshes every 15 seconds.
"ask ai" fails	Window 5 not running or no key — skip Act 7 or start the mock provider.
Port already in use (3000 / 4000 / 27017)	Close old windows from a previous run, or restart the PC before the demo.
Every page shows HTTP 500; console window prints "Cannot find module ./611.js"	The dev cache is corrupted (npm run build was run while the dev server ran). Ctrl+C in Window 3, then: cd D:\projects\cn\frontend; Remove-Item -Recurse -Force .next; npm run dev
Agent prints "heartbeat failed … deadline exceeded"	The PC changed network, so its address changed. Check ipconfig and restart the agent with --server http://<new address>:4000.
PC-2 cannot open http://192.168.0.105:3000	On this PC: Wi-Fi set to Private and lan-firewall.ps1 -Server -Agent run as Administrator; check the address with ipconfig.
PC-2: "could not join — the token may have expired"	Tokens last 15 minutes. Click "register a pc" again and copy the new command.
PC-2 enrolled but topology stays dotted	Both sides must run with --userspace; allow the Windows network prompt; on this PC the -Agent firewall rule opens UDP 51820.
Approved test shows "timed out"	The lab was not running when you approved — start it (Act 5) and approve a new test.

Resetting for another run

1. Ctrl+C in every window.
2. Use a new database name in `control-plane\.env` (e.g. nexlink_demo2), or switch back to nexlink for your normal data.
3. Delete the agent folders `networking-agent\bin\demo-a` and `demo-b` (they contain private keys; git ignores them anyway).

Documents to have ready

Document	Use it for
README on GitHub	Opening slide: logo, features, diagrams
docs/architecture.md	Deeper architecture questions
docs/network-protocols.md	Enrollment, request signing, telemetry, NAT/relay design
docs/threat-model.md	Security questions
docs/NexLink-Project-Report.docx / .pdf	Full project report with test results

Document	Use it for
docs/two-pc-test.md	Connecting a second physical PC (quick userspace path + WireGuard app path)

7. Printable checklist

- ☐ git pull; npm install in control-plane and frontend
- ☐ control-plane\.env uses MONGODB_URI=…/nexlink_demo (fresh database)
- ☐ Decided on AI: Groq key ready / mock / hidden
- ☐ Rehearsed once, then reset the database
- ☐ Word, Docker Desktop and old terminals closed
- ☐ Window 1: npm run dev:mongo running
- ☐ Window 2: control plane listening on 4000
- ☐ Window 3: console ready on http://localhost:3000
- ☐ Window 4: Administrator PowerShell in networking-agent\bin
- ☐ Window 5 (optional): diagnostic service on 8090
- ☐ Browser zoom 80–90%; README and report open in tabs
- ☐ Act 2: demo-a and demo-b enrolled
- ☐ Act 4b: simulation page opens; normal + control plane down scenarios rehearsed
- ☐ Act 5: lab running, topology line lime, agent console streaming
- ☐ Act 5b (optional): firewall script run once; PC-2 on the same Wi-Fi opens the portal by LAN address
- ☐ Act 6: approved test verified successful
- ☐ Act 8: revoke shown, audit log shown, lab stopped